МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации

Кафедра Систем Информационной Безопасности

Лабораторная работа № 4

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Функциональная декомпозиция, глобальная сумма и критические секции»

Работу выполнил

Студент гр. 4404

Ионов И.А.

Казань, 2026

1. Цель работы

Освоить функциональную декомпозицию с общей (глобальной) переменной, критические секции и убедиться на опыте, к чему приводит состояние гонки.

2. Постановка задач

Задача 6. Используя функциональную декомпозицию, переписать задачу 5 для вычисления значения по формуле (Σx + Σy + Σz) / (3N), где N - количество точек. Использовать глобальную сумму и критические секции.

Примечание. В тексте пособия формула записана как «∑x∑y∑z/3N»; знаки между суммами при подготовке пособия, по-видимому, потерялись. Принято естественное толкование: (Σx + Σy + Σz) / (3N), то есть среднее значение всех координат; оно равно среднему от трёх координат центра из задачи 5.

3. Реализация

Файл lab4/Task6.java. Функции: Σx, Σy и Σz считаются в разных потоках (функциональная декомпозиция). При числе потоков 3k каждая функция дополнительно делится между k потоками по данным.

Глобальная сумма globalSum - общая переменная. Критическая секция реализована блоком synchronized(LOCK) (аналог #pragma omp critical). Реализовано три режима: **critical** - каждый поток считает локальную сумму и один раз добавляет её к глобальной в критической секции; **critical-each** - критическая секция на каждое слагаемое; **race** - без синхронизации (демонстрация гонки).

Результат сравнивается с последовательным вычислением; выводятся время и расхождение.

Сборка и запуск (подробнее в README папки lab4):

```
cd lab4
javac Task6.java GenPoints.java
java Task6 sample.txt 3 all
java GenPoints 5000000 points.txt
java Task6 points.txt 6 all
```

4. Ход работы и скриншоты

Рабочий код:

```
import java.io.*;
import java.util.Arrays;

/**
 * Задача 6. Значение по формуле (Σx + Σy + Σz) / (3N), N - число точек.
 * Функциональная декомпозиция: задача делится по ФУНКЦИЯМ - одна функция суммирует x, другая y, третья z.
 * Если потоков больше трёх (кратно 3), каждая функция дополнительно делится между потоками по данным.
 * Результаты потоков складываются в ОБЩУЮ (глобальную) сумму внутри критической секции.
 *
 * Запуск: java Task6 <файл> [потоки=3] [режим=all|critical|critical-each|race]
 *   critical      - поток считает локальную сумму и один раз добавляет её к глобальной в критической секции
 *   critical-each - критическая секция на КАЖДОЕ слагаемое (корректно, но медленно)
 *   race          - без синхронизации (демонстрация состояния гонки: результат неверный)
 */
public class Task6 {
    static double[] xs, ys, zs;
    static int n;
    static final Object LOCK = new Object();   // аналог #pragma omp critical
    static double globalSum;                    // общая (разделяемая) сумма

    static void readFile(String name) throws IOException {
        int cap = 1024;
        xs = new double[cap]; ys = new double[cap]; zs = new double[cap];
        n = 0;
        try (BufferedReader r = new BufferedReader(new FileReader(name), 1 << 16)) {
            String line;
            int lineNo = 0;
            while ((line = r.readLine()) != null) {
                lineNo++;
                line = line.trim();
                if (line.isEmpty()) continue;
                String[] p = line.split("[\\s;,]+");
                if (p.length < 3) throw new IOException("Line " + lineNo + ": expected 3 numbers");
                if (n == cap) {
                    cap *= 2;
                    xs = Arrays.copyOf(xs, cap); ys = Arrays.copyOf(ys, cap); zs = Arrays.copyOf(zs, cap);
                }
                xs[n] = Double.parseDouble(p[0]);
                ys[n] = Double.parseDouble(p[1]);
                zs[n] = Double.parseDouble(p[2]);
                n++;
            }
```

```
        }
    }

    static double sequentialFormula() {
        double s = 0;
        for (int i = 0; i < n; i++) s += xs[i];
        for (int i = 0; i < n; i++) s += ys[i];
        for (int i = 0; i < n; i++) s += zs[i];
        return s / (3.0 * n);
    }

    /** Возвращает значение формулы; время работы потоков записывает в elapsed[0]. */
    static double run(String mode, int threads, double[] elapsed) throws InterruptedException {
        globalSum = 0;
        final int k = threads / 3;              // потоков на одну функцию
        Thread[] ts = new Thread[threads];
        for (int w = 0; w < threads; w++) {
            final double[] data = (w % 3 == 0) ? xs : (w % 3 == 1) ? ys : zs;   // какая функция: Σx, Σy
или Σz
            final int slice = w / 3;
            final int from = (int) ((long) n * slice / k);
            final int to = (int) ((long) n * (slice + 1) / k);
            ts[w] = new Thread(() -> {
                switch (mode) {
                    case "critical" -> {
                        double local = 0;
                        for (int i = from; i < to; i++) local += data[i];
                        synchronized (LOCK) { globalSum += local; }
                    }
                    case "critical-each" -> {
                        for (int i = from; i < to; i++)
                            synchronized (LOCK) { globalSum += data[i]; }
                    }
                    case "race" -> {
                        for (int i = from; i < to; i++) globalSum += data[i];   // без синхронизации!
                    }
                    default -> throw new IllegalArgumentException(mode);
                }
            });
        }
        long t0 = System.nanoTime();
        for (Thread t : ts) t.start();
        for (Thread t : ts) t.join();
        elapsed[0] = (System.nanoTime() - t0) / 1e9;
        return globalSum / (3.0 * n);
    }

    public static void main(String[] args) throws Exception {
        if (args.length < 1) { System.err.println("Usage: java Task6 <file> [threads] [mode]"); return; }
        int threads = args.length > 1 ? Integer.parseInt(args[1]) : 3;
        String mode = args.length > 2 ? args[2] : "all";
        if (threads < 1) { System.err.println("threads >= 1"); return; }
```

```
        if (threads % 3 != 0) {
            int fixed = (threads / 3 + 1) * 3;
            System.out.println("Note: number of threads must be a multiple of 3 (one group per x, y, z);
using " + fixed);
            threads = fixed;
        }
        readFile(args[0]);
        if (n == 0) { System.err.println("File has no points"); return; }
        System.out.printf("Read %d points, threads: %d%n", n, threads);

        long t0 = System.nanoTime();
        double ref = sequentialFormula();
        System.out.printf("%-14s result = %.9f  time %.3f ms%n", "sequential", ref,
(System.nanoTime() - t0) / 1e6);

        String[] modes = mode.equals("all") ? new String[]{"critical", "critical-each", "race"} : new
String[]{mode};
        double[] el = new double[1];
        for (String m : modes) {
            double best = 1e9, res = 0;
            for (int r = 0; r < 3; r++) {           // 3 запуска, берём лучшее время
                res = run(m, threads, el);
                best = Math.min(best, el[0]);
            }
            System.out.printf("%-14s result = %.9f  time %.3f ms  |diff vs seq| = %.2e  global sum =
%.6f%n",
                    m, res, best * 1e3, Math.abs(res - ref), globalSum);
        }
    }
}
```

Скриншот 1. Запуск: `java Task6 sample.txt`

```
Read 8 points, threads: 6
sequential     result = 4,875000000  time 0,012 ms
critical       result = 4,875000000  time 0,867 ms  |diff vs seq| = 0,00e+00  global sum = 117,000000
critical-each  result = 4,875000000  time 0,665 ms  |diff vs seq| = 0,00e+00  global sum = 117,000000
race           result = 4,875000000  time 0,980 ms  |diff vs seq| = 0,00e+00  global sum = 117,000000
```

Пояснение. Проверка на файле из 8 точек: глобальная сумма равна 117 (26 + 38 + 53), результат 117 / 24 = 4.875 совпадает с расчётом вручную. На столь малом объёме гонка не проявляется.

5. Ответы на вопросы

Вопрос. Зачем нужна критическая секция при добавлении к глобальной сумме?

Ответ. Операция globalSum += v состоит из чтения, сложения и записи. Если два потока выполняют её одновременно, один из них может перезаписать результат другого

(потерянное обновление). Критическая секция гарантирует, что в каждый момент ею владеет один поток, и результат получается верным.

Вопрос. Почему critical быстрее, чем critical-each?

Ответ. В critical поток входит в секцию один раз за всю работу, а в critical-each - на каждый элемент, то есть миллионы раз. Захват и освобождение блокировки дороги, к тому же потоки ожидают друг друга, и параллелизм практически теряется. Поэтому число точек синхронизации нужно минимизировать: считать локально и объединять результат один раз.

Вопрос. Что показывает режим race?

Ответ. Состояние гонки: без синхронизации результат неверен и меняется от запуска к запуску. Ошибки нет в тексте программы, она проявляется только при определённом чередовании потоков, поэтому такие ошибки трудно обнаружить тестированием.

6. Выводы

- Формула вычислена с помощью функциональной декомпозиции; результат совпадает с последовательной версией и с ручным расчётом.
- Доступ к общей глобальной сумме должен быть защищён критической секцией; без неё результат неверен (гонка).
- Синхронизация имеет цену: критическую секцию следует использовать как можно реже (локальные суммы + одно объединение).